3. Build a Vector Database Using FAISS and Perform Similarity-Based Document Retrieval

Aim

To build a vector database using FAISS (Facebook AI Similarity Search) and perform efficient similarity-based document retrieval using text embeddings.

Algorithm

1. Install and import the required libraries, including FAISS and Sentence Transformers.
2. Create a collection of sample documents for retrieval.
3. Load a pre-trained Sentence Transformer model.
4. Generate numerical vector embeddings for all documents.
5. Normalize the document embeddings to unit length.
6. Create a FAISS index using the embedding dimension.
7. Add the document embeddings to the FAISS index.
8. Accept a search query from the user and generate its embedding.
9. Normalize the query embedding and search the FAISS index for the nearest vectors.
10. Retrieve and display the most relevant documents with their similarity scores.

Code

```
# Install required libraries
!pip install -q sentence-transformers faiss-cpu

# Import libraries
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# Sample documents
documents = [
    "Artificial Intelligence makes machines intelligent.",
    "Machine Learning allows computers to learn from data.",
```

```
    "Python is a programming language.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing understands human language.",
    "Computer Vision helps machines understand images.",
    "Cloud Computing provides online computing services."
]

# Load Sentence Transformer model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Generate document embeddings
embeddings = model.encode(
    documents,
    convert_to_numpy=True
)

# Convert embeddings into float32
embeddings = np.asarray(embeddings, dtype=np.float32)

# Normalize embeddings for cosine similarity
faiss.normalize_L2(embeddings)

# Get embedding dimension
dimension = embeddings.shape[1]

# Create FAISS index
index = faiss.IndexFlatIP(dimension)

# Add document vectors to FAISS
```

```
index.add(embeddings)

print("FAISS Vector Database Created Successfully!")
print("Total Documents:", index.ntotal)

# Get search query
query = input("\nEnter your search query: ")

# Generate query embedding
query_embedding = model.encode(
    [query],
    convert_to_numpy=True
)

query_embedding = np.asarray(
    query_embedding, dtype=np.float32
)

# Normalize query vector
faiss.normalize_L2(query_embedding)

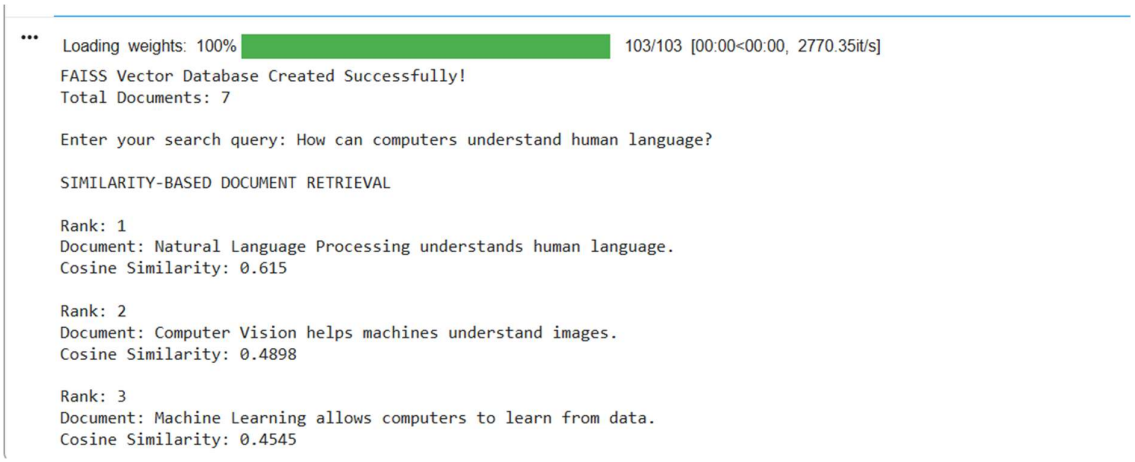
# Search top 3 similar documents
k = min(3, len(documents))

scores, indices = index.search(query_embedding, k)

# Display retrieved documents
print("\nSIMILARITY-BASED DOCUMENT RETRIEVAL")
```

```
for rank, (idx, score) in enumerate(
    zip(indices[0], scores[0]), start=1
):
    print(f"\nRank: {rank}")
    print("Document:", documents[idx])
    print("Cosine Similarity:", round(float(score), 4))
```

Output:



Result

Thus, a vector database was successfully created using FAISS, and similarity-based document retrieval was performed using normalized text embeddings and cosine similarity.